Problema dos Jarros D'Água resolvido por Algoritmos de Busca sem Informação

Enzio Gabriel, Jonas Leal

Tecnologia em Análise e Desenvolvimento de Sistemas (ADS) – Módulo III
Instituto Federal do Piauí (IFPI) – Campus Picos
Disciplina de Inteligência Artificial – Prof. Rogério Figueredo de Sousa

Resumo. *Este trabalho apresenta a formulação e a resolução do Problema dos Jarros D'Água, com jarros de 4 L e 3 L e objetivo de obter exatamente 2 L em qualquer um deles, por meio de três algoritmos de busca sem informação: busca em largura (BFS), busca em profundidade (DFS) e aprofundamento iterativo (IDDFS). Os algoritmos foram implementados em Python e comparados quanto ao número de nós gerados e expandidos, ao custo da solução e ao tempo de execução. Para o estado inicial (0, 0), BFS e IDDFS encontraram a solução ótima, com 4 ações, enquanto a DFS encontrou uma solução de 6 ações. Os resultados indicam que, com custo unitário por ação, a BFS é adequada a espaços de estados pequenos e o IDDFS é preferível quando a memória é um limitante.*

Abstract. *This paper presents the formulation and solution of the Water Jug Problem (4 L and 3 L jugs, with the goal of measuring exactly 2 L in either jug) using three uninformed search algorithms: breadth-first search (BFS), depth-first search (DFS) and iterative-deepening depth-first search (IDDFS). The algorithms were implemented in Python and compared by the number of generated and expanded nodes, solution cost and running time. From the initial state (0, 0), BFS and IDDFS found the optimal 4-action solution, while DFS returned a 6-action solution. The results indicate that, with unit action costs, BFS is suitable for small state spaces, whereas IDDFS is preferable when memory is a limiting factor.*

1. Introdução

Este relatório apresenta a formulação e a resolução do Problema dos Jarros D'Água utilizando três algoritmos de busca sem informação: busca em largura (BFS), busca em profundidade (DFS) e busca com aprofundamento iterativo (IDDFS). Há dois jarros, um de 4 litros e outro de 3 litros, inicialmente vazios, e uma fonte de água inesgotável. O objetivo é obter exatamente 2 litros em qualquer um dos jarros.

O programa foi implementado em Python 3 (arquivo `jarros.py`, apenas com a biblioteca padrão). Ele recebe um estado inicial (por padrão, (0, 0)), executa os três algoritmos e imprime a árvore de busca resultante, destacando o caminho até a solução, a sequência de ações e as métricas de cada execução: nós gerados, nós expandidos,

custo da solução e tempo médio. O programa também exporta as árvores no formato Graphviz (.dot) (Graphviz, 2026) e gera uma tabela comparativa para vários estados iniciais (opção `--bench`).

2. Formulação do Problema

Seguindo a orientação de fazer apenas as distinções necessárias para garantir uma solução válida, o problema foi formulado assim:

2.1 Representação dos estados

Um estado é um par ordenado **(a, b)**, em que **a** é a quantidade de litros no jarro de 4 L ($0 \le a \le 4$) e **b** é a quantidade no jarro de 3 L ($0 \le b \le 3$). Só importa a quantidade de água em cada jarro; não é preciso modelar a água desperdiçada, a posição da fonte ou o tempo. O espaço de estados tem $5 \times 4 = 20$ estados possíveis. No código, os estados são tuplas (a, b), e as capacidades são as constantes `CAP_A = 4` e `CAP_B = 3`.

2.2 Estado inicial e teste de objetivo

- **Estado inicial:** fornecido pelo usuário na linha de comando (no enunciado, (0, 0), com os dois jarros vazios). O programa valida que $0 \le a \le 4$ e $0 \le b \le 3$ e encerra com uma mensagem de erro caso contrário.
- **Teste de objetivo:** um estado (a, b) é objetivo se $a = 2$ ou $b = 2$.
- **Custo de caminho:** cada ação custa 1; o custo da solução é o número de ações.

2.3 Operadores (ações)

As quatro ações do enunciado (encher, esvaziar, completar um jarro com o outro e despejar o conteúdo de um jarro dentro do outro) aplicam-se a cada um dos dois jarros, o que resulta em oito operadores. As duas formas de transferência são distinguidas pelo que acontece com a água: em “completar”, o destino enche e ainda sobra água na origem; em “despejar”, todo o conteúdo da origem cabe no destino e a origem fica vazia. As pré-condições dessas duas formas são mutuamente exclusivas, de modo que um mesmo par de jarros nunca gera dois sucessores iguais.

Tabela 1. Operadores do problema

Operador	Efeito sobre (a, b)	Pré-condição
Encher 4L	(a, b) → (4, b)	$a < 4$
Encher 3L	(a, b) → (a, 3)	$b < 3$
Esvaziar 4L	(a, b) → (0, b)	$a > 0$
Esvaziar 3L	(a, b) → (a, 0)	$b > 0$
Completar 3L com 4L	$t = 3 - b$; $(a - t, 3)$	$a > 0$, $b < 3$ e $a > 3 - b$
Completar 4L com 3L	$t = 4 - a$; $(4, b - t)$	$b > 0$, $a < 4$ e $b > 4 - a$
Despejar 4L em 3L	(a, b) → $(0, a + b)$	$a > 0$ e $a \le 3 - b$
Despejar 3L em 4L	(a, b) → $(a + b, 0)$	$b > 0$ e $b \le 4 - a$

Como cada operador só é gerado quando a sua pré-condição é satisfeita, ações que não alteram o estado (por exemplo, encher um jarro já cheio ou esvaziar um jarro vazio) nunca são geradas, o que evita laços triviais.

3. Algoritmos Implementados

Os três algoritmos seguem as definições clássicas de busca sem informação: a busca em largura expande os nós por nível, a busca em profundidade segue um ramo até o fim antes de retroceder e o aprofundamento iterativo combina as duas ideias repetindo a busca com limites crescentes (Cormen et al., 2009; Korf, 1985; Russell and Norvig, 2020).

Tabela 2. Resumo dos algoritmos

Algoritmo	Estrutura / estratégia	Controle de repetições	Teste de objetivo
Largura (BFS)	Fila FIFO; expande nível a nível	Conjunto de estados visitados, atualizado na geração do nó	Ao gerar o nó
Profundidade (DFS)	Pilha LIFO; segue um ramo até o fim	Conjunto de estados visitados, consultado ao retirar o nó da pilha	Ao retirar o nó da pilha
Aprofundamento iterativo (IDDFS)	DFS limitada (recursiva) repetida com limite 0, 1, 2, …	Evita apenas ciclos no caminho atual	Ao entrar no nó

Como a BFS testa o objetivo já na geração do nó, ela encontra a solução sem precisar expandir o nó-objetivo. Nos três algoritmos, o nó-objetivo não é contado como expandido. Os nós repetidos também são criados e contados como gerados (aparecem na árvore marcados com `[repetido]`), mas não são expandidos.

Os sucessores são gerados sempre na mesma ordem (encher 4L, encher 3L, esvaziar 4L, esvaziar 3L, completar 3L com 4L, completar 4L com 3L, despejar 4L em 3L, despejar 3L em 4L), o que torna os resultados reproduzíveis. Na DFS, os filhos são empilhados em ordem inversa para que o primeiro sucessor seja o primeiro a ser explorado.

Como métricas, o programa registra: nós gerados, nós expandidos, custo da solução (igual à profundidade, pois cada ação custa 1) e tempo médio de execução, medido com `time.perf_counter` (Python Software Foundation, 2026) como média de 2000 execuções (1000 no modo `--bench`). Internamente, também é registrado o tamanho máximo da fronteira (a fila na BFS e a pilha na DFS); no IDDFS, esse valor é aproximado por limite + 1, que é a profundidade máxima da recursão.

4. Resultados para o estado inicial (0, 0)

As árvores abaixo foram produzidas pelo programa (`python jarros.py 0 0`). Nós sem rótulo e sem filhos foram gerados, mas a busca terminou antes de expandi-los.

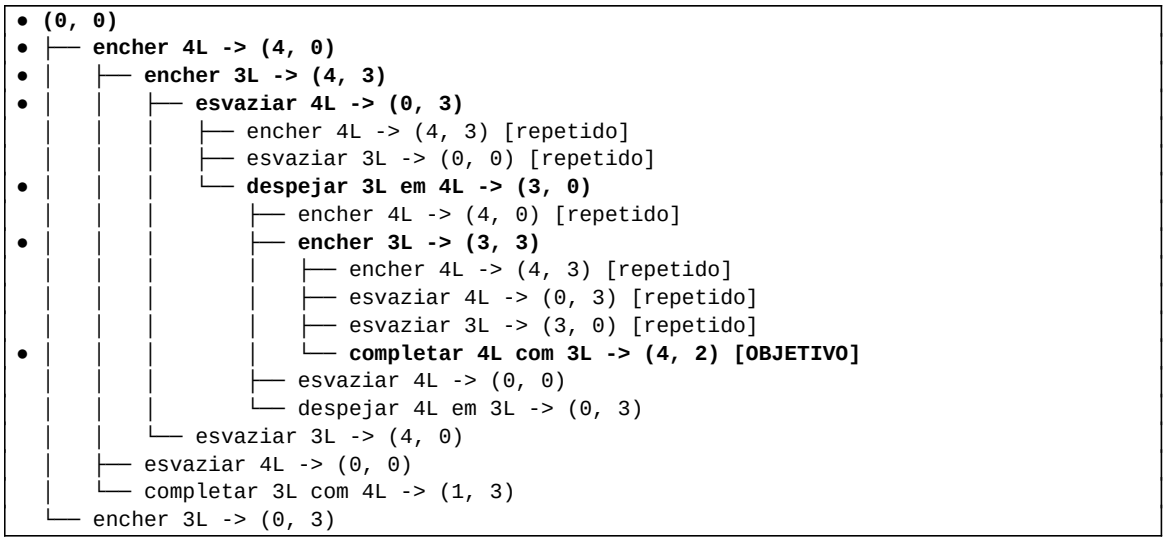
● nó do caminho da solução `[repetido]` já visitado `[limite]` profundidade-limite (IDDFS) **`[OBJETIVO]`**

4.1 Busca em Largura

```
● (0, 0)
  ├── encher 4L -> (4, 0)
  │   ├── encher 3L -> (4, 3)
  │   │   ├── esvaziar 4L -> (0, 3) [repetido]
  │   │   └── esvaziar 3L -> (4, 0) [repetido]
  │   ├── esvaziar 4L -> (0, 0) [repetido]
  │   └── completar 3L com 4L -> (1, 3)
  │       ├── encher 4L -> (4, 3) [repetido]
  │       ├── esvaziar 4L -> (0, 3) [repetido]
  │       ├── esvaziar 3L -> (1, 0)
  │       │   ├── encher 4L -> (4, 0) [repetido]
  │       │   ├── encher 3L -> (1, 3) [repetido]
  │       │   ├── esvaziar 4L -> (0, 0) [repetido]
  │       │   └── despejar 4L em 3L -> (0, 1)
  │       └── despejar 3L em 4L -> (4, 0) [repetido]
● └── encher 3L -> (0, 3)
      ├── encher 4L -> (4, 3) [repetido]
      ├── esvaziar 3L -> (0, 0) [repetido]
●     └── despejar 3L em 4L -> (3, 0)
          ├── encher 4L -> (4, 0) [repetido]
●         ├── encher 3L -> (3, 3)
          │   ├── encher 4L -> (4, 3) [repetido]
          │   ├── esvaziar 4L -> (0, 3) [repetido]
          │   ├── esvaziar 3L -> (3, 0) [repetido]
●         │   └── completar 4L com 3L -> (4, 2) [OBJETIVO]
          ├── esvaziar 4L -> (0, 0) [repetido]
          └── despejar 4L em 3L -> (0, 3) [repetido]
```

Caminho da solução — 4 ações (ótima)
(0,0) → (0,3) → (3,0) → (3,3) → (4,2)
Encher 3L, despejar 3L em 4L, encher 3L, completar 4L com 3L. A BFS gerou **27 nós** e expandiu **8**.

4.2 Busca em Profundidade



Caminho da solução — 6 ações (válida, mas não ótima)
(0,0) → (4,0) → (4,3) → (0,3) → (3,0) → (3,3) → (4,2)
Como a DFS segue sempre o primeiro sucessor, ela chega a (0,3) por um desvio de três ações (encher 4L, encher 3L, esvaziar 4L) e só então percorre o mesmo trecho final da BFS. Gerou **19 nós** e expandiu **6**. Os nós sem rótulo e sem filhos, como o "encher 3L → (0,3)" da raiz, ficaram na pilha sem serem explorados.

4.3 Aprofundamento Iterativo

O IDDFS encontrou a solução no limite de profundidade 4, após 5 iterações. A árvore abaixo é a da última iteração (limite 4); os contadores do programa, por outro lado, acumulam todas as iterações. A Tabela 3 mostra o trabalho de cada iteração.

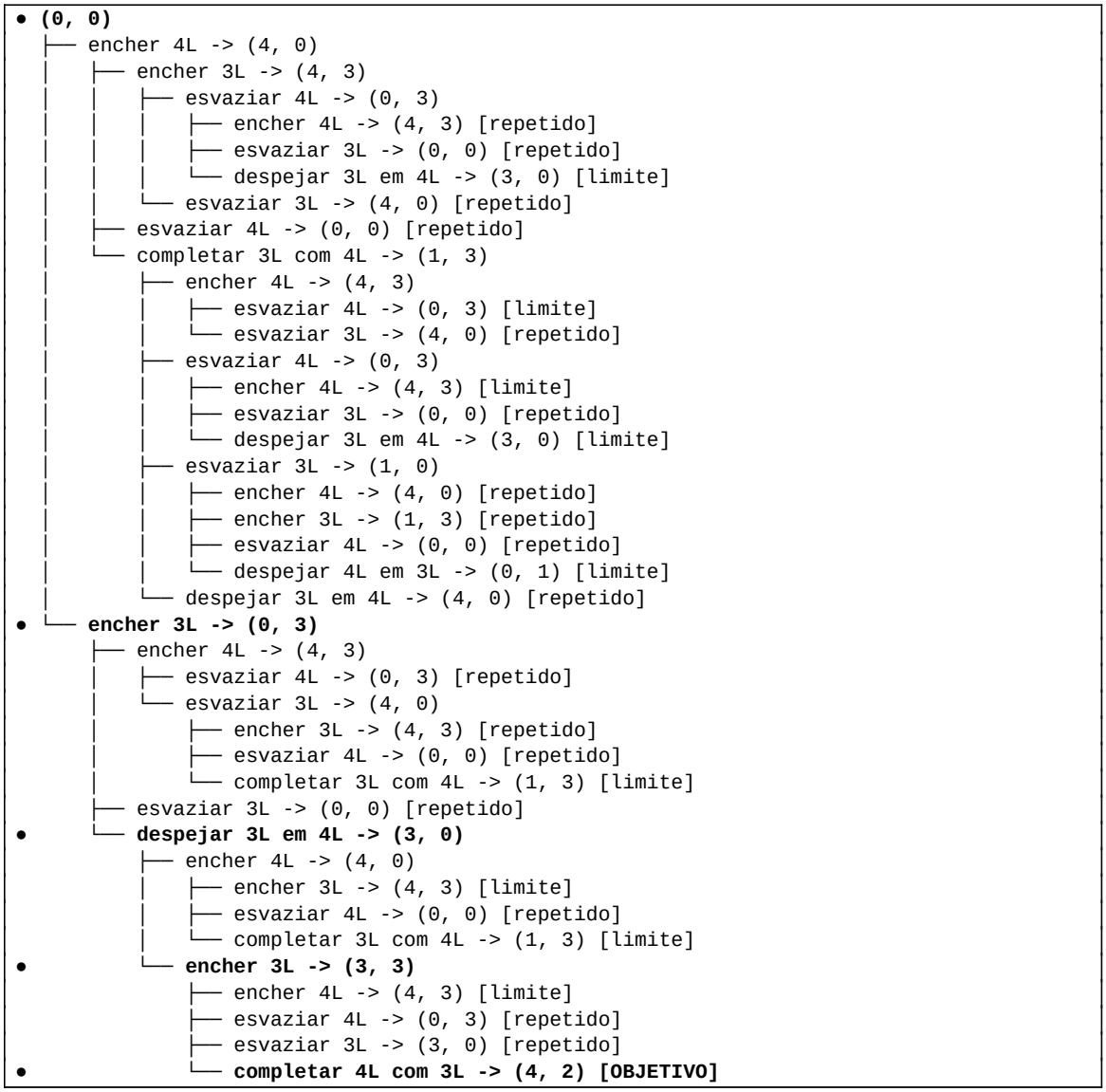


Tabela 3. Trabalho por iteração do IDDFS (total: 75 gerados e 25 expandidos)

Limite	Nós gerados	Nós expandidos	Achou solução?
0	1	0	Não
1	3	1	Não
2	9	3	Não
3	21	7	Não
4	41	14	Sim

Caminho da solução — 4 ações
(0,0) → (0,3) → (3,0) → (3,3) → (4,2), o mesmo da BFS.

5. Análise Comparativa

Tabela 4. Comparação para o estado inicial (0, 0). Tempos variam conforme a máquina e a execução. * No IDDFS, limite + 1.

Algoritmo	Expandidos	Gerados	Custo	Fronteira máx.	Tempo médio (µs)
Busca em Largura	8	27	4	3	≈ 34
Busca em Profundidade	6	19	6	10	≈ 19
Aprofundamento Iterativo	25	75	4	5 *	≈ 72

- **Expansões:** a DFS foi a que menos expandiu (6), mas porque o primeiro ramo que seguiu levou ao objetivo; a solução que ela achou não é a mais curta. A BFS expandiu 8 nós e o IDDFS expandiu 25, pois refaz o trabalho das iterações anteriores. Em nós gerados, a BFS (27) gerou mais que a DFS (19), porque cada expansão gera até oito sucessores, inclusive repetidos.
- **Qualidade da solução:** BFS e IDDFS são ótimos aqui, porque o custo de cada ação é 1; ambos acharam 4 ações. A DFS achou 6 ações, o que mostra que ela não garante otimalidade.
- **Tempo:** acompanhou o número de nós gerados: o IDDFS foi cerca de três vezes mais lento que a BFS e a DFS, que ficaram próximas.
- **Memória:** em teoria, a BFS guarda a fronteira inteira, com custo O(b^d); a DFS e o IDDFS guardam essencialmente o caminho atual, com custo O(b·m) e O(b·d), respectivamente. Neste problema, porém, o espaço tem apenas 20 estados e a diferença não aparece nas medições: a fronteira máxima da DFS (10) foi maior que a da BFS (3), pois a DFS empilha todos os sucessores de cada nó expandido e, além disso, mantém um conjunto de estados visitados (assim como a BFS). Só o IDDFS guarda apenas o caminho atual. A vantagem teórica de memória da DFS e do IDDFS só se torna relevante em espaços de estados muito maiores.

Para conferir que o comportamento não depende só do estado (0, 0), o programa também foi executado com outros estados iniciais (Tabela 5). Em todos os casos, BFS e IDDFS acharam soluções do mesmo custo, e a DFS achou soluções de custo igual ou maior: a partir de (3, 3), por exemplo, BFS e IDDFS resolvem com 1 ação, enquanto a DFS usa 9. O estado (1, 2) já é objetivo (b = 2): os três algoritmos retornam custo 0 gerando apenas o nó inicial.

Tabela 5. Nós expandidos (exp.) e custo da solução para outros estados iniciais

Estado inicial	BFS exp.	BFS custo	DFS exp.	DFS custo	IDDFS exp.	IDDFS custo
(0, 0)	8	4	6	6	25	4
(4, 0)	9	5	6	5	39	5

Estado inicial	BFS exp.	BFS custo	DFS exp.	DFS custo	IDDFS exp.	IDDFS custo
(0, 3)	6	3	8	7	13	3
(3, 0)	3	2	9	6	4	2
(4, 3)	7	4	8	8	18	4
(1, 1)	1	1	11	6	1	1
(3, 3)	1	1	9	9	1	1

6. Conclusão

O Problema dos Jarros D'Água foi formulado com estados **(a, b)**, em que **a** é o volume de água no jarro de 4 L e **b** o volume no jarro de 3 L, oito operadores de custo 1 e teste de objetivo **a = 2 ou b = 2**. A Figura 1 mostra a árvore de busca construída pela busca em largura (BFS) a partir do estado inicial (0, 0). Ela foi gerada pelo programa `jarros.py` (python `jarros.py` 0 0 --algo bfs --dot arvore) e desenhada com o Graphviz.

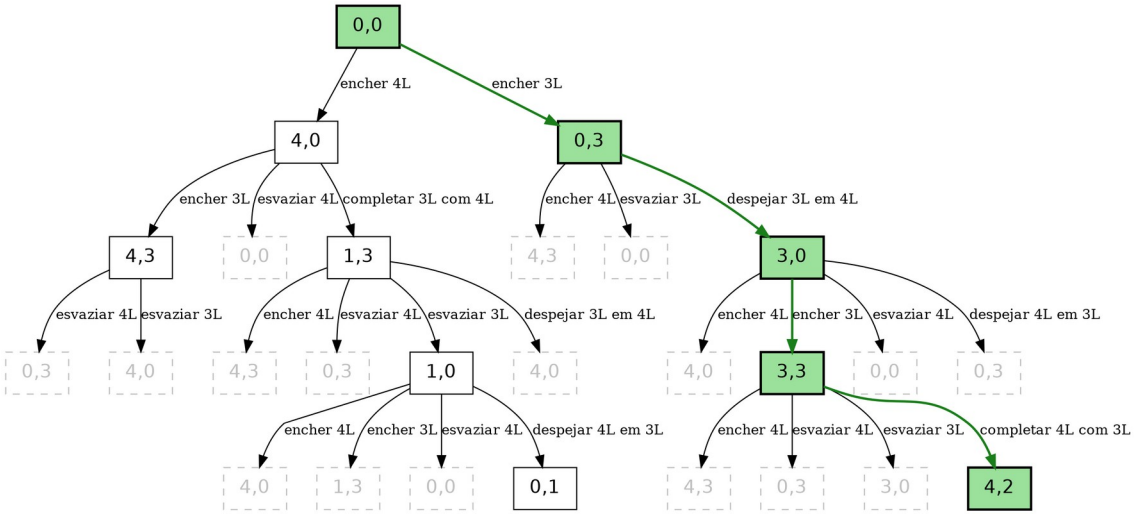


Figura 1. Árvore de busca em largura (BFS) para o estado inicial (0, 0). Verde: caminho da solução; cinza tracejado: estado repetido; branco: estado novo.

6.1 Explicação da árvore, do início ao fim

A árvore é lida de cima para baixo, nível a nível, na mesma ordem em que a BFS (fila FIFO) a constrói. Os sucessores de cada nó são sempre gerados na ordem: encher 4L, encher 3L, esvaziar 4L, esvaziar 3L, completar 3L com 4L, completar 4L com 3L, despejar 4L em 3L e despejar 3L em 4L. Um estado já visitado é marcado como repetido: ele aparece na árvore (e conta como nó gerado), mas não é expandido.

Nível 0 --- raiz (0, 0). Os dois jarros estão vazios e o estado não é objetivo, pois nenhum jarro tem 2 L. Como não há água para esvaziar, completar ou despejar, só duas ações se aplicam: encher 4L, que leva a (4, 0), e encher 3L, que leva a (0, 3). Os dois filhos são estados novos e entram na fila.

Nível 1 --- (4, 0) e (0, 3). A BFS expande primeiro (4, 0), que gera (4, 3) ao encher 3L, (0, 0) ao esvaziar 4L (repetido, pois é a raiz) e (1, 3) ao completar 3L com 4L: o jarro de 3 L recebe 3 L e sobra 1 L no de 4 L. Depois expande (0, 3), que é o primeiro nó do caminho da solução. Dele, encher 4L e esvaziar 3L levam a (4, 3) e (0, 0), ambos repetidos, e despejar 3L em 4L leva ao estado novo (3, 0).

Nível 2 --- (4, 3), (1, 3) e (3, 0). Os três são expandidos nessa ordem. De (4, 3) só saem estados repetidos. De (1, 3) sai um único estado novo, (1, 0), ao esvaziar o jarro de 3 L. De (3, 0) sai o estado novo (3, 3), ao encher 3L; os demais filhos são repetidos. Nenhum dos nós dos níveis 0 a 2 satisfaz a = 2 ou b = 2, então a busca continua.

Níveis 3 e 4 --- (1, 0), (3, 3) e o objetivo. Primeiro se expande (1, 0), que gera apenas um estado novo, (0, 1); esse nó fica na fila e não chega a ser expandido, porque a busca termina antes. Em seguida se expande (3, 3). Encher 4L, esvaziar 4L e esvaziar 3L levam a estados repetidos, mas completar 4L com 3L gera (4, 2): o jarro de 4 L tem 3 L e só comporta mais 1 L, então sobram 2 L no jarro de 3 L. Como a BFS aplica o teste de objetivo no momento da geração do nó, (4, 2) é reconhecido na hora (b = 2) e a busca para, sem expandi-lo. Ao todo foram gerados 27 nós e expandidos 8: (0,0), (4,0), (0,3), (4,3), (1,3), (3,0), (1,0) e (3,3).

Percorrendo o caminho verde da raiz até o nó objetivo, obtém-se a sequência de ações da solução:

Tabela 6. Sequência de ações da solução ótima (BFS)

Passo	Ação	Estado	O que acontece
0	estado inicial	(0, 0)	Os dois jarros estão vazios.
1	encher 3L	(0, 3)	O jarro de 3 L é cheio na fonte.
2	despejar 3L em 4L	(3, 0)	Os 3 L cabem inteiros no jarro de 4 L; o de 3 L fica vazio.
3	encher 3L	(3, 3)	O jarro de 3 L é cheio de novo.
4	completar 4L com 3L	(4, 2)	O jarro de 4 L recebe só 1 L e fica cheio; sobram 2 L no de 3 L. Objetivo.

Como a BFS expande a árvore por níveis e cada ação custa 1, o primeiro objetivo encontrado, na profundidade 4, é o de menor custo. Isso se confirma na árvore: nenhum estado dos níveis 0 a 3 é objetivo, logo não existe solução com 3 ações ou menos.

6.2 Comparação com a busca em profundidade e o aprofundamento iterativo

Tabela 7. Resumo comparativo das três buscas para o estado inicial (0, 0)

Algoritmo	Gerados	Expandidos	Custo	Solução
Busca em Largura (BFS)	27	8	4	Ótima
Busca em Profundidade (DFS)	19	6	6	Válida, mas não ótima
Aprofundamento Iterativo (IDDFS)	75	25	4	Ótima (limite 4, 5 iterações)

A DFS expandiu menos nós (6) porque seguiu primeiro o ramo encher 4L → encher 3L → esvaziar 4L, que a levou a (0, 3) por um desvio de três ações. Só depois percorreu o mesmo trecho final da BFS, e por isso sua solução tem 6 ações. O IDDFS repete a busca com limites de 0 a 4 e, por isso, expande 25 nós no total, mas devolve o mesmo caminho ótimo da BFS.

6.3 Considerações finais

Os três algoritmos resolveram o problema. A busca em largura e o aprofundamento iterativo retornaram a solução ótima, com 4 ações a partir de (0, 0), enquanto a busca em profundidade retornou uma solução mais longa, de 6 ações, ainda que com menos expansões nesse caso. O IDDFS combina a otimalidade da BFS com o uso de memória da DFS, ao custo de repetir expansões, o que aparece no maior número de nós expandidos e no maior tempo de execução.

O espaço de estados tem apenas 20 estados, então a vantagem de memória da DFS e do IDDFS não aparece nas medições. Já a diferença de qualidade das soluções aparece com clareza: nos sete estados iniciais testados, BFS e IDDFS sempre acharam soluções do mesmo custo, e a DFS achou soluções de custo igual ou maior, chegando a 9 ações a partir de (3, 3), caso em que BFS e IDDFS resolvem com 1 ação. Conclui-se que, para problemas em que todas as ações têm o mesmo custo e se deseja a solução mais curta, a BFS é a escolha adequada quando o espaço é pequeno, e o IDDFS passa a ser preferível quando a memória se torna um limitante (Russell and Norvig, 2020; Korf, 1985).

7. Como executar o programa

```
$ python jarros.py     # estado inicial (0, 0), os três algoritmos
$ python jarros.py 0 0 # estado inicial (a=0, b=0)
$ python jarros.py 4 1 --algo bfs     # só um algoritmo (bfs | dfs | iddfs)
$ python jarros.py 0 0 --dot arvore   # exporta arvore_bfs.dot, ...
$ python jarros.py 0 0 --sem-arvore   # só caminho e métricas
$ python jarros.py --bench    # tabela comparativa
```

Os arquivos .dot podem ser convertidos em imagem com o Graphviz, por exemplo: `dot -Tpng arvore_bfs.dot -o arvore_bfs.png`.

Referências

Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2009). Introduction to Algorithms. MIT Press, Cambridge, MA, 3rd edition.

Graphviz (2026). Graphviz documentation. Available at: https://graphviz.org/documentation/. Accessed on: Oct. 14, 2026.

Korf, R. E. (1985). Depth-first iterative-deepening: An optimal admissible tree search. Artificial Intelligence, 27(1):97–109.

Python Software Foundation (2026). The Python Standard Library: time — Time access and conversions. Available at: https://docs.python.org/3/library/time.html. Accessed on: Oct. 14, 2026.

Russell, S. J. and Norvig, P. (2020). Artificial Intelligence: A Modern Approach. Pearson, Hoboken, NJ, 4th edition.